

Keebox Encryption & Key Architecture Specification

KBKey, KMKey, AES-256-GCM, key representation, recovery, and implementation rules

1. Purpose

This document defines the agreed Keebox encryption architecture for KBoxes and the key entities used to protect them. It covers KBKey, KMKey, key structure, allowed characters, the conversion layer that turns the presentation format into raw cryptographic key bytes, the encryption hierarchy, server storage expectations, restore behavior, and recommended libraries.

2. Encryption Model

Keebox uses symmetric authenticated encryption. KCredentialBox and KNoteBox data are encrypted on the client before they are synchronized to the backend. The backend therefore stores encrypted KBox payloads rather than the raw private contents.

AES-256-GCM is the selected encryption primitive. AES-256 uses a 32-byte key, while GCM provides both confidentiality and integrity protection.

3. Key Hierarchy

Keebox uses two key entities with separate responsibilities.

KMKey -> encrypts/decrypts KBKeys
KBKey -> encrypts/decrypts KCredentialBox and KNoteBox data

4. KBKey - Keebox Key

Every Keebox user has exactly one KBKey. It is created for that user and remains stable throughout the normal lifetime of the account. The KBKey is the user's direct encryption and decryption key for all KCredentialBox and KNoteBox content.

KBKey responsibilities

- Directly encrypts KCredentialBox data using AES-256-GCM.
- Directly decrypts KCredentialBox data using AES-256-GCM.
- Directly encrypts KNoteBox data using AES-256-GCM.
- Directly decrypts KNoteBox data using AES-256-GCM.
- Is unique to one Keebox user.
- Is not stored as plaintext in the backend database.

5. KMKey - Keebox Master Key

KMKey is the server-level Keebox Master Key. Its purpose is not to encrypt user KBoxes directly. Its purpose is to protect KBKeys while they are stored by the backend.

KMKey responsibilities

- Directly encrypts KBKeys before they are persisted by the backend.
- Directly decrypts KBKeys when an authenticated user must receive their KBKey.
- Is a server secret and must be stored outside the application database.
- Must be protected through environment secret management or a dedicated secret-management system.

6. Official Key Presentation Format

KBKey and KMKey have different identifying prefixes but use the same two-segment Base64url structure.

```
KBKey: KBK-<22-character segment>-<21-character segment>
KMKey: KMK-<22-character segment>-<21-character segment>
```

The segments together contain the 43-character unpadded Base64url encoding of exactly 32 cryptographically random bytes. The prefix and structural delimiter are presentation metadata and are not part of the raw key bytes.

7. Base64url Characters and Structural Delimiter

The encoded payload uses the Base64url alphabet shown below. Padding characters are not allowed.

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789-_  

```

Because the hyphen is valid Base64url data, implementations must not parse a formatted key with `split('-')`. The structural delimiter is always at index 26: prefix indexes 0-3, first segment 4-25, delimiter 26, and second segment 27-47. The total formatted length is exactly 48 characters.

8. Raw Cryptographic Key Construction

The formatted key is a reversible presentation encoding. The conversion layer validates the structure and recovers the original AES-256 key bytes.

```
Generate exactly 32 bytes with a cryptographically secure random generator
-> encode as unpadded Base64url -> exactly 43 characters
-> split encoded text at 22 characters -> 22 + 21
-> format as KBK-<segment 1>-<segment 2>
-> parse positionally, join segments, decode -> exactly 32 bytes
```

No hashing or KDF is performed. The generated 32 bytes are the AES-256 key; Base64url encoding and segmentation do not derive or alter the key material.

9. Effective Entropy

A uniformly random 32-byte key has the full 256 bits of entropy required by this design. Base64url encoding is reversible and does not reduce that entropy. Implementations must reject non-canonical values by decoding, re-encoding without padding, and requiring exact equality with the joined 43-character payload.

10. KBKey Storage on the Backend

The backend must never store a user's KBKey in plaintext. Before database persistence, the server uses the KMKey to encrypt the complete formatted KBKey. The resulting encrypted value is stored with the user record or an associated key record.

```
KBKey
-> AES-256-GCM using KMKey
-> encrypted KBKey
-> backend database
```

When a successfully authenticated client requires the KBKey, the backend retrieves the encrypted KBKey, decrypts it with the KMKey, and returns the KBKey to the client over an authenticated HTTPS connection.

11. KBox Encryption Flow

The Flutter client obtains the user's KBKey, converts it to the raw 32-byte cryptographic key, and uses that key directly with AES-256-GCM to encrypt KCredentialBox and KNoteBox payloads before local persistence or remote synchronization.

```
KCredentialBox / KNoteBox plaintext
-> AES-256-GCM using KBKey
-> ciphertext + nonce + authentication tag
-> encrypted local storage and encrypted backend synchronization
```

12. Nonce Requirements

AES-GCM requires a unique nonce for every encryption operation performed with the same key. The nonce does not need to be secret, but it must never be reused for two different plaintexts under the same KBKey or KMKey. The implementation should rely on the cryptography library's secure nonce generation or on a cryptographically secure random generator.

13. Backup and Restore Relationship

KBoxes remain encrypted before they are sent to the backend. The backend stores encrypted KBox payloads. Because the user's KBKey is itself recoverably stored in encrypted form using the KMKey, a user who logs out, uninstalls the application, or changes devices can authenticate again, obtain the same KBKey from the backend, download their encrypted KBoxes, and decrypt them locally.

This design deliberately prioritizes straightforward account recovery and device restoration over a strict zero-knowledge server model. The backend can recover KBKeys because it possesses the KMKey.

14. Key Lifecycle Rules

- One KBKey exists per user.
- A KBKey is generated once and does not normally change.
- A KMKey is a server-level secret and is not user-specific.
- KBoxes are never encrypted directly with the KMKey.
- KBKeys are never persisted as plaintext on the backend.

- The KMKey must never be stored in the same database that stores encrypted KBKeys.
- Key values must never be written to application logs, analytics, crash reports, or debug output.
- All network transfer of KBKeys and encrypted KBox data must occur over HTTPS.

15. Recommended Libraries

For Flutter/Dart, the recommended package is **cryptography**. It provides modern authenticated encryption primitives including AES-GCM and is suitable for client-side cryptographic operations. The package should be used for KBox encryption and decryption, nonce generation where supported, and authenticated-cipher handling.

For Django/Python, the recommended library is **cryptography**. It provides AES-GCM through the AESGCM primitive and is appropriate for encrypting and decrypting KBKeys using the server KMKey.

16. Final Architecture

The agreed encryption hierarchy is:

```

KMKey (KMK-...) | | AES-256-GCM v Encrypted KBKey in backend database | | backend decrypts
when authenticated user requires it v KBKey (KBK-...) | | AES-256-GCM v KCredentialBox /
KNoteBox ciphertext

```

The KBK- and KMK- formats are therefore human-readable presentation structures for direct AES-256 key material, while the dedicated conversion layer is responsible for validating those structures and extracting the exact 32 bytes used by the cryptographic primitive.